ADR 0004 - Base model choice and baseline ladder

Status: Proposed **Date**: 2026-06-23 **Author**: Ronin **Related**: docs/ROADMAP.md, docs/BENCHMARKING.md, ADR 0005, docs/WRITEUP_INFO.md

---

Context

BAYMAX is intended to become a locally hostable personal-task agent. That means the project cannot judge success only by whether a hosted frontier model performs well. It needs a baseline ladder:

1. Scripted responses prove the eval harness and fake adapters work
2. A naive hosted-model agent gives a realistic capability baseline
3. A small local model becomes the Phase 2 fine-tuning target

The base model choice affects scenario design, SFT formatting, cost reporting, and the final portfolio claim. Choosing too large a model first slows iteration. Choosing too weak a model risks spending Phase 2 debugging model incapability instead of measuring whether structured training improves tool-use behavior.

Decision

Phase 1 keeps the eval harness model-agnostic. The first baseline is the scripted control run in `results/phase1/scripted/v1-baseline.json`; it costs `0.0` and validates the harness rather than model intelligence.

After the scripted control, Marv's naive agent should be evaluated through the same `AgentResponse` boundary and fake adapter environment. Hosted models may be used for baseline comparison, but they are not the target deployment model.

For Phase 2 local fine-tuning, the starting target is a Qwen 2.5-class instruct model around 1.5B parameters, unless Phase 2 benchmarking identifies a better locally hostable candidate. The choice optimizes for:

```
local serving feasibility
low inference cost
fast SFT iteration
adequate instruction following
structured JSON/tool-call learnability
```

The inference backend remains swappable per ADR 0005. This ADR picks the initial local-model direction, not a permanent provider lock.

Reasoning

why chosen - the scripted baseline tells us whether Ronin's harness is correct before blaming a model. After that, a hosted naive-agent baseline gives us a reference point for what "good enough" looks like on the same 50 scenarios. Qwen-size local models are small enough to iterate on without turning every experiment into an infrastructure project, while still being plausible for structured instruction-following and tool-call formatting. This keeps the project honest: first prove the benchmark, then compare models, then fine-tune.

Alternatives considered

Alternative A - GPT-only baseline and deployment

Use GPT-class hosted models for both baseline and final behavior.

why rejected - useful as a comparison point, but it does not satisfy the locally hostable thesis. It also makes cost and privacy weaker parts of the final story.

Alternative B - Start with a larger local model

Use a 7B+ local model as the first training and serving target.

why rejected - stronger raw capability, but slower iteration and higher hardware pressure. The Phase 2 question is whether the pipeline and task-specific data improve behavior; a smaller model gives faster feedback.

Alternative C - Pick no model until after data prep

Keep Phase 2 model selection completely open.

why rejected - SFT export format, prompt templates, context limits, and serving assumptions need an approximate model family. Fully deferring the choice would push too much uncertainty into Phase 2.

Alternative D - Train before scripted baseline is complete

Begin SFT data generation before the 50-scenario harness is finished.

why rejected - bad benchmark data creates bad training data. The Phase 1 harness has to be stable before it becomes the source of SFT examples.

Consequences

Positive

- Separates harness correctness from model capability
- Keeps local hosting central to the project thesis
- Gives Phase 2 a concrete but swappable starting model family
- Supports cost comparison between scripted, hosted, and local runs

Negative

- A 1.5B-class model may underperform hosted baselines substantially
- Some failures may reflect model size rather than agent architecture
- Future prompt/template changes may be needed if the chosen local model has format quirks

Neutral

- Backend abstraction remains owned by Marv's inference layer
- Ronin's eval output records backend/model name so comparisons can coexist

---

Open questions / follow-ups

- Which exact Qwen checkpoint and quantization will Phase 2 use?
- What hosted model becomes the naive baseline comparison?
- What train/eval split prevents leakage from scripted scenarios into SFT evaluation?
- What metric must improve to justify the fine-tuned model: cost, latency, task success, or argument accuracy?

References

- docs/ROADMAP.md
- docs/BENCHMARKING.md
- ADR 0005 - Inference backend
- `results/phase1/scripted/v1-baseline.json`
- Qwen 2.5 model family documentation

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.